

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 12
Operating Systems
Topic: Producer, Consumer and the Other Tools (Track A - Linux)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Laboratory Objectives 3

2 PART 1: THE PRODUCER-CONSUMER PROBLEM **4**

2.1 Program Implementation (`prodcons.c`) 4

2.2 Execution and Trace Output 5

2.3 Answers to Required Observation Questions 5

3 PART 2: CONDITION VARIABLES **7**

3.1 Program Implementation (`condvar.c`) 7

3.2 Execution Trace 7

3.3 Answers to Required Observation Questions 7

4 PART 3: THE READERS-WRITERS PROBLEM **9**

4.1 Program Implementation (`readers.c`) 9

4.2 Execution Trace 9

4.3 Answers to Required Observation Questions 10

5 PART 4: DEADLOCK DETECTION AND PREVENTION **11**

5.1 Deadlock Analysis and Coffman Conditions 11

5.2 The Deadlock Program and Inverted Lock Ordering 11

5.3 The Corrected Program (`deadlock.c`) 11

5.4 Answers to Required Observation Questions 12

6 CONCLUSIONS **14**

REFERENCES **15**

1 INTRODUCTION AND GOAL

1.1 Introduction

In concurrent system architectures, inter-thread coordination extends far beyond basic mutual exclusion around simple variables. Real-world operating systems and multi-threaded applications rely on classic coordination paradigms to manage pipelines, stream buffers, asymmetric data access, and asynchronous event notifications.

Four archetypal synchronization patterns constitute the foundation of concurrent software engineering:

a) **The Bounded-Buffer (Producer-Consumer) Problem:** Coordinating concurrent threads that produce and consume data items through a fixed-capacity ring buffer without buffer overflow or underflow.

b) **Condition Variables:** Providing efficient, stateless synchronization that allows threads to suspend execution until a complex predicate becomes true, avoiding wasteful CPU busy-waiting (spinning).

c) **The Readers-Writers Problem:** Managing asymmetric concurrent access to shared data structures where multiple readers may safely inspect data simultaneously, but writers demand strict mutual exclusion.

d) **Deadlock Detection and Prevention:** Analyzing the four Coffman conditions that lead to permanent system freezes, and enforcing strict global resource hierarchies to eliminate circular waits.

1.2 Laboratory Objectives

The specific goals of this laboratory work include:

- Implementing Dijkstra's two-semaphore and mutex solution for the bounded-buffer problem (`prodcons.c`) and analyzing thread synchronization under asymmetric producer/consumer speeds.
- Exploring POSIX condition variables (`pthread_cond_wait`, `pthread_cond_signal`) in `condvar.c`, contrasting their stateless nature with stateful semaphores, and analyzing why loop re-evaluation (`while`) is mandatory under Mesa semantics.
- Implementing the reader-preference Readers-Writers algorithm (`readers.c`) using dual semaphores, and evaluating the problem of writer starvation.
- Reproducing deterministic circular deadlock (`deadlock.c`), modeling the deadlock with a Resource Allocation Graph (RAG), and applying a global lock hierarchy to eliminate deadlocks.

2 PART 1: THE PRODUCER-CONSUMER PROBLEM

2.1 Program Implementation (`prodcons.c`)

The bounded-buffer model utilizes a circular array of size $N = 4$ carrying $M = 8$ sequential integer items. Synchronization is coordinated via two counting semaphores and one mutual exclusion lock:

- `sem_t empty`: Initialized to $SIZE = 4$, tracking available empty slots.
- `sem_t full`: Initialized to 0, tracking populated slots ready for extraction.
- `pthread_mutex_t lock`: Serializes updates to buffer indices (`in`, `out`) and counter variables.

```c
/* prodcons.c - producer and consumer around a buffer of 4 slots. */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;

sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty); /* block if buffer is full */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put␣%d␣(in␣buffer:␣%d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full); /* signal new available item */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full); /* block if buffer is empty */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("␣␣␣␣␣␣take␣%d␣(in␣buffer:␣%d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty); /* signal free slot */
        usleep(300000);   /* slow consumer simulation */
    }
    return NULL;
}

int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
```

```
48      pthread_t p, c;
49      pthread_create(&c, NULL, consumer, NULL);
50      pthread_create(&p, NULL, producer, NULL);
51      pthread_join(p, NULL);
52      pthread_join(c, NULL);
53      sem_destroy(&empty);
54      sem_destroy(&full);
55      return 0;
56  }
```

2.2 Execution and Trace Output

```
1   $ gcc -Wall -pthread -o prodcons prodcons.c
2   $ ./prodcons
3   put 1 (in buffer: 1)
4   put 2 (in buffer: 2)
5   put 3 (in buffer: 3)
6   put 4 (in buffer: 4)
7         take 1 (in buffer: 3)
8   put 5 (in buffer: 4)
9         take 2 (in buffer: 3)
10  put 6 (in buffer: 4)
11        take 3 (in buffer: 3)
12  put 7 (in buffer: 4)
13        take 4 (in buffer: 3)
14  put 8 (in buffer: 4)
15        take 5 (in buffer: 3)
16        take 6 (in buffer: 2)
17        take 7 (in buffer: 1)
18        take 8 (in buffer: 0)
```

Figure 2.1: Figure 2.1 – Bounded-buffer execution trace showing producer throttled at capacity 4

2.3 Answers to Required Observation Questions

a) **In `prodcons.c`, which call makes the producer stop when the buffer is full?**

The invocation of **`sem_wait(&empty);`** at line 21. When the producer rapidly outputs items 1 through 4 while the consumer sleeps, the initial 4 permits in `empty` are decremented to 0. When the producer loops to item 5, calling `sem_wait(&empty)` on a semaphore whose value is 0 immediately blocks the producer thread. The producer remains suspended until the consumer removes item 1 and calls `sem_post(&empty)`.

b) **When the program starts, does the consumer run first or does it wait? Which line makes it do that?**

The consumer **waits** (blocks) immediately at startup. The line responsible is **`sem_wait(&full);`** at line 35. Because the semaphore `full` is initialized to 0 in `main()`, attempting to decrement it blocks the consumer until the producer inserts the first item and signals completion via `sem_post(&full)`.

c) **What would happen if the producer locked the mutex BEFORE calling `sem_wait(&empty)`?**

This would cause an immediate, catastrophic **Deadlock**:

1) When the buffer fills up ($count = 4, empty = 0$), the producer would execute `pthread_mutex_lock(&l` and acquire the mutex.

2) The producer then calls `sem_wait(&empty)`. Because $empty = 0$, the producer is put to sleep by the kernel *while continuing to hold* `lock`.

3) The consumer attempts to process items to free up slots. However, the consumer must execute `pthread_mutex_lock(&lock)` before extracting data.

4) Because the producer holds the mutex, the consumer blocks waiting for `lock`.

5) The producer is waiting for the consumer to increment `empty`, while the consumer is waiting for the producer to release `lock`. Neither thread can proceed.

This demonstrates the fundamental rule of synchronization: **never block on a semaphore while holding a mutex needed by the signaling thread**.

3 PART 2: CONDITION VARIABLES

3.1 Program Implementation (`condvar.c`)

POSIX condition variables allow threads to sleep until a specific application condition is met. In `condvar.c`, the signaling problem from Lab 11 is re-implemented using a mutex, a condition variable, and an explicit boolean condition flag:

```c
/* condvar.c - thread signaling via mutex and condition variable */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
int ready = 0; /* explicit condition state */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1; /* update condition predicate */
    printf("A: message written\n");
    pthread_cond_signal(&changed); /* wake waiting reader */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) { /* loop re-check predicate */
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock); /* atomically unlock & sleep */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

3.2 Execution Trace

```
$ gcc -Wall -pthread -o condvar condvar.c
$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

3.3 Answers to Required Observation Questions

a) **Which variable remembers that the message is ready?**

The user-level variable **`int ready`**. A condition variable itself possesses **no internal memory or counter**. If `pthread_cond_signal()` is dispatched when no thread is actively blocked inside

`pthread_cond_wait()`, the signal vanishes permanently. Therefore, the application state must be explicitly preserved in a program variable (`ready`), protected by the associated mutex.

b) **In `reader()`, replace `while (!ready)` with `if (!ready)`. Does it still work?**

Yes, in this simple single-producer, single-consumer program, replacing `while` with `if` still functions correctly. Because there is only one reader thread and no competing threads modifying `ready`, when the reader is awakened by the writer, `ready` is guaranteed to be 1.

c) **Why does POSIX recommend `while` instead of `if`?**

POSIX systems and modern operating system kernels follow **Mesa Monitor Semantics**, necessitating a `while` loop for two primary reasons:

- **Spurious Wakeups:** Under Linux and Unix kernels, `pthread_cond_wait()` can legally unblock and return even when no thread has signaled the condition variable. This can occur due to signal deliveries, kernel futex internal wakeups, or multiprocessing race resolutions.
- **Multi-Consumer Races:** In systems with multiple waiting threads, signaling with `pthread_cond_br` or waking multiple consumers means that another thread might acquire the mutex first, process the item, and reset `ready = 0`. When the second thread eventually re-acquires the mutex, the condition is no longer satisfied. The `while` loop forces the thread to re-verify the predicate before proceeding.

4 PART 3: THE READERS-WRITERS PROBLEM

4.1 Program Implementation (`readers.c`)

The Readers-Writers problem coordinates concurrent access to shared data where read operations can occur simultaneously, but write operations require exclusive access:

- `sem_t room`: Initialized to 1, acts as an exclusive lock on the shared memory room.

- `sem_t rlock`: Initialized to 1, protects updates to the active reader counter `readers`.

```c
/* readers.c - multiple concurrent readers, exclusive writers */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;
int readers = 0;
sem_t rlock; /* protects readers counter */
sem_t room;  /* 1 = room free for a writer */

void *reader(void *arg) {
    long id = (long)arg;

    sem_wait(&rlock);
    if (++readers == 1)
        sem_wait(&room); /* first reader claims the room */
    sem_post(&rlock);

    printf("reader␣%ld␣reads␣%d␣(readers␣inside:␣%d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)
        sem_post(&room); /* last reader releases the room */
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room); /* exclusive access */
    value += 10;
    printf("writer␣%ld␣writes␣%d␣(alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}
```

4.2 Execution Trace

```
$ gcc -Wall -pthread -o readers readers.c
$ ./readers
writer 1 writes 10 (alone)
```

```
4 reader 1 reads 10 (readers inside: 1)
5 reader 2 reads 10 (readers inside: 2)
6 reader 3 reads 10 (readers inside: 3)
7 writer 2 writes 20 (alone)
```

Figure 4.1: Figure 4.1 – Concurrent readers executing together while writers execute in isolation

4.3 Answers to Required Observation Questions

a) **Which two lines allow multiple readers to read at the same time?**

The entry and exit threshold checks:

- Entry check (Line 22): `if (++readers == 1) sem_wait(&room);`
- Exit check (Line 30): `if (--readers == 0) sem_post(&room);`

Only the very first reader that enters performs `sem_wait(&room)`, locking out writers. Subsequent readers increment `readers` and bypass the semaphore entirely, entering concurrently. When leaving, intermediate readers simply decrement the counter; only the final reader out calls `sem_post(&room)`, relinquishing the room back to waiting writers.

b) **Why does the `readers` counter need its own lock `rlock`?**

The operations `++readers` and `--readers` are non-atomic read-modify-write sequences. If multiple reader threads arrive simultaneously without `rlock`, a race condition on `readers` could lead to two threads both seeing `readers == 1` and attempting to acquire `room` twice, or a missed decrement where `readers` never reaches 0, causing waiting writers to be permanently locked out.

c) **What happens if readers keep arriving one after another, with no gap?**

Writer Starvation: This implementation favors readers. If new reader threads arrive before previous readers have departed, the active count `readers` remains strictly ≥ 1. Consequently, `sem_post(&room)` is never called, and any waiting writer threads remain indefinitely blocked in the kernel queue, completely starved of execution time.

5 PART 4: DEADLOCK DETECTION AND PREVENTION

5.1 Deadlock Analysis and Coffman Conditions

A deadlock occurs when a set of concurrent processes are permanently blocked because each process holds a resource and waits for another resource held by another process. For a deadlock to occur, four Coffman conditions must hold simultaneously:

a) **Mutual Exclusion:** Resources cannot be shared simultaneously.

b) **Hold and Wait:** Processes hold allocated resources while requesting new ones.

c) **No Preemption:** Resources cannot be forcibly revoked from holding processes.

d) **Circular Wait:** A closed chain of processes exists where each process waits for a resource held by the next.

5.2 The Deadlock Program and Inverted Lock Ordering

In the uncorrected version of `deadlock.c`, Thread `one` requests Mutex A then Mutex B, while Thread `two` requests Mutex B then Mutex A:

```
/* deadlock_bug.c - buggy inverted locking order */
void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B); /* blocks waiting for B */
    ...
}

void *two(void *arg) {
    pthread_mutex_lock(&B);
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&A); /* blocks waiting for A */
    ...
}
```

When compiled and monitored with the GNU `timeout` command, the program hangs permanently until terminated by the watchdog timer:

```
$ gcc -Wall -pthread -o deadlock_bug deadlock_bug.c
$ timeout 4 ./deadlock_bug
one: I have A, I want B
two: I have B, I want A
$ echo "exit code: $?"
exit code: 124
```

Exit code 124 confirms that the program was forcibly killed after 4 seconds of deadlock.

5.3 The Corrected Program (`deadlock.c`)

To eliminate circular wait, a strict global lock hierarchy ($A \prec B$) was established. Thread `two` was modified to acquire Mutex A first, followed by Mutex B:

```c
/* deadlock.c - fixed: both threads lock A first, then B */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&A); /* same order as thread one */
    printf("two: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("done\n");
    return 0;
}
```

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 4 ./deadlock
one: I have A, I want B
one: I have both
two: I have A, I want B
two: I have both
done
$ echo "exit code: $?"
exit code: 0
```

5.4 Answers to Required Observation Questions

a) **Draw the wait circle on paper: thread one holds ..., wants ...; thread two holds ..., wants ...:**

The Resource Allocation Graph (RAG) forms a closed directed cycle:

$$\text{Thread 1} \xrightarrow{\text{holds}} \text{Lock A} \xleftarrow{\text{wants}} \text{Thread 2} \xrightarrow{\text{holds}} \text{Lock B} \xleftarrow{\text{wants}} \text{Thread 1} \tag{5.1}$$

In graphical circular wait representation:

Thread 1 ⟶ requests Lock B ⟶ held by Thread 2 ⟶ requests Lock A ⟶ held by Thread 1

Because neither thread can proceed without the other releasing its held resource, both threads enter an unresolvable circular wait.

b) **If you delete `sleep(1)`, the program might occasionally finish without hanging. Why is that worse than hanging every time?**

Removing `sleep(1)` transforms a deterministic defect into a non-deterministic **Heisenbug**:

- When a bug manifests deterministically (hanging 100% of the time), automated test suites immediately fail, pinpointing the defect before code enters production.
- When the timing window is narrow, the code may run successfully 9,999 times out of 10,000 during local development and CI testing. The bug slips silently into production, where it triggers catastrophic system freezes under heavy real-world server workloads, making root-cause diagnostics extremely expensive and difficult.

c) **Fix the program: change only thread two so it takes the locks in the same order as thread one (A, then B). Run it again. Does it finish now?**

Yes, the corrected program completes in 2.01 seconds and prints `done` with exit code `0`. By forcing both threads to acquire Mutex A before Mutex B, Thread 2 cannot acquire any lock until Thread 1 completely releases Mutex A. The circular wait condition is mathematically broken, completely immunizing the system against deadlock.

6 CONCLUSIONS

In this laboratory work, advanced multi-threaded synchronization primitives and classic concurrency patterns were systematically implemented and evaluated under Linux.

The bounded-buffer model established that counting semaphores cleanly regulate multi-slot queues, while highlighting the critical necessity of ordering semaphore checks outside mutex locks to prevent deadlocks. Condition variables illustrated the mechanics of stateless thread suspension and confirmed why Mesa monitor semantics mandate predicate re-checking via `while` loops to guard against spurious wakeups.

The readers-writers investigation exposed how prioritization decisions affect fairness, demonstrating that reader-favoring policies inevitably lead to writer starvation under sustained loads. Finally, deadlocks were dissected through Coffman's conditions, proving that establishing a strict global lock hierarchy is an infallible, mathematically sound strategy for eliminating circular waits in multi-threaded software.

REFERENCES

[1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, Wiley, 10th Edition, 2018.

[2] Michael Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.

[3] W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, Addison-Wesley, 3rd Edition, 2013.

[4] Department of Software Engineering and Automation, *Operating Systems Lab 12: Producer, Consumer and the Other Tools*, FCIM, Technical University of Moldova, 2026.